Session Token

A **session token** is a temporary identifier issued by the server after a user successfully logs in. It is used to maintain an authenticated session between the client and server without requiring the user to re-enter credentials on every request.

It is one of the most common mechanisms for **stateful authentication** in web applications.

1. Core Idea

Instead of sending username/password repeatedly:

```
None
1. User logs in once

2. Server verifies credentials

3. Server creates session

4. Server sends session token to client

5. Client sends token with future requests
```

The token acts as proof that the user is already authenticated.

2. How Session Token Works

Step-by-Step Flow

```
None
Client → Login Request (email/password)


Server:
✔ Validate credentials
✔ Create session in DB / Redis
✔ Generate session token


Server → Set-Cookie: session_token=abc123


Client stores cookie automatically
```

For future requests:

```
None
Client → Sends session_token cookie


Server:

✔ Looks up token

✔ Finds active session

✔ Authorizes request
```

3. Architecture Flow

```
None
User Browser

     ↓

Login Request

     ↓

Application Server

     ↓

Create Session Record

     ↓

Redis / DB

     ↓

Return Session Token Cookie
```

Next requests:

```
None
Browser → Cookie → Server → Validate Session → Response
```

4. Session Token Storage

On Client Side

Usually stored in:

```
None
HTTP Cookie (recommended)

Secure Cookie

HttpOnly Cookie

SameSite Cookie
```

Best for browsers.

Can also be stored in:

```
None
Local Storage

Session Storage

Mobile App Secure Storage
```

On Server Side

Session state is stored in:

```
None
Redis

Database

Memory Store

Distributed Cache
```

Example:

```
None
session_token: abc123


{
 user_id: 101,
 role: admin,
 created_at: ...
 expires_at: ...
}
```

5. Why Session Tokens Are Used

✔ Better Security

```
None
Password not sent every request
```

✔ Stateful Authentication

Server can track:

```
None
Logged-in devices

Expiry

Logout

Permission changes
```

✔ Easy Revocation

```
None
Delete session record → token instantly invalid
```

6. Session Token vs JWT

Feature	Session Token	JWT
Stateful	Yes	No (usually)
Stored on Server	Yes	No
Easy Logout	Yes	Harder
Horizontal Scaling	Needs shared store	Easier
Size	Small	Larger
Revocation	Easy	Complex

7. Session Lifecycle

Login

```
None

Create token + session
```

Active Usage

```
None

Validate token each request
```

Expiry

```
None

Auto-expire after inactivity / TTL
```

Logout

```
None

Delete session record

Clear cookie
```

8. Security Best Practices

✔ Use Secure Cookies

```
None

Secure      → HTTPS only

HttpOnly    → JS cannot read

SameSite    → CSRF protection
```

✔ Rotate Tokens

Generate new token periodically.

✔ Set Expiration

```
None
15 min / 1 day / 7 days
```

Depends on product.

✔ Store in Redis

Fast validation:

```
None
O(1) lookup
```

✔ Detect Suspicious Activity

```
None
IP change

Device change

Multiple locations
```

9. Scaling Session Tokens

For multiple servers:

```
User → Load Balancer → App Servers
                    ↘
                     Redis Cluster
```

All servers validate sessions from shared Redis.

10. Common Problems

Session Hijacking

Attacker steals cookie.

Mitigation:

```
HTTPS

Secure cookies

Short expiry

Device binding
```

Session Fixation

Attacker forces known token before login.

Mitigation:

```
None
Regenerate token after login
```

Too Many Sessions

Use:

```
None
Per-user session limit

Old session invalidation
```

11. Interview Answer Example

How would you manage login sessions?

```
None
After authentication, generate a random session token,

store session data in Redis with TTL,

send token as HttpOnly secure cookie,

validate token on each request,

delete token on logout.
```

12. Real World Examples

Used heavily by:

```
None
Banking apps

E-commerce websites

Admin dashboards

Enterprise apps

Traditional web apps
```

13. Session Token in Chat App

```
None
Login → Session token

WebSocket handshake includes token

Server validates token before opening socket
```

14. One-Line Summary

A session token is a server-issued temporary identifier that allows authenticated users to stay logged in securely, with session state stored on the server for validation, expiry, and logout control.